

Machine Learning para Finanzas

Clase 2

César Núñez Cuevas

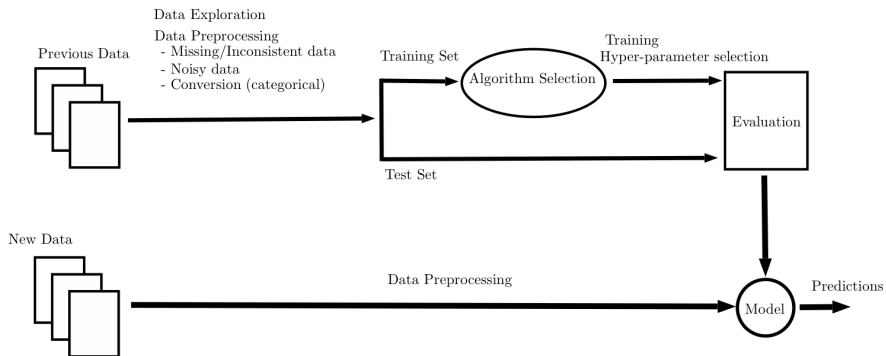
`cnunezc@fen.uchile.cl`



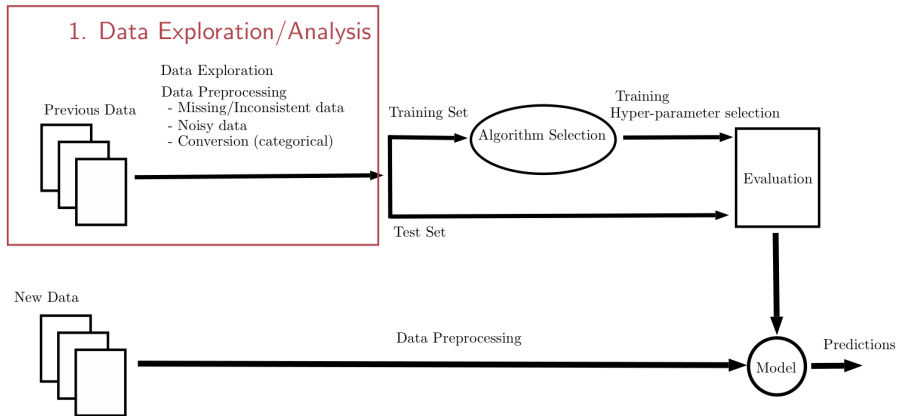
UNIVERSIDAD

Finis Terrae

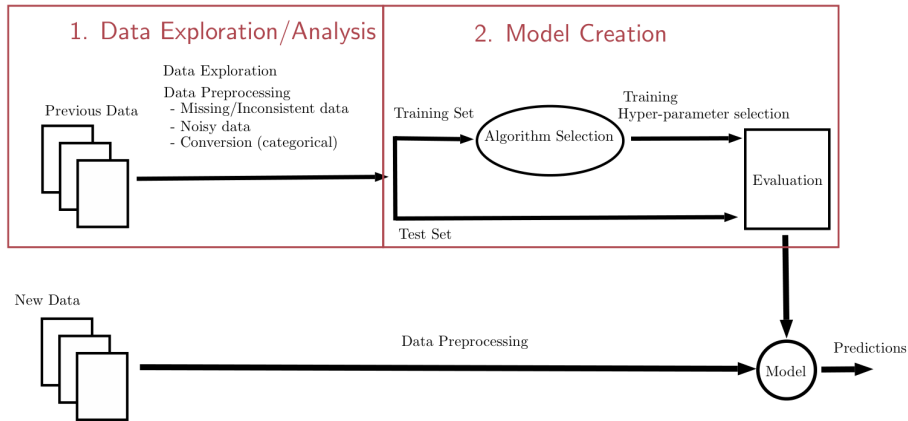
Workflow



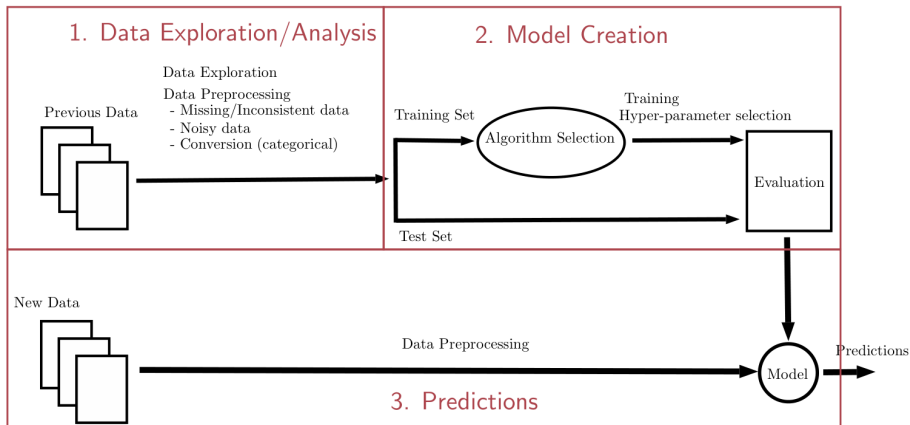
Workflow



Workflow



Workflow



Estructura de Datos: El DataFrame

- **Definición:** Estructura bidimensional, mutable y heterogénea. Es la base de pandas.
- **Componentes:**
 - ① **Index:** Etiquetas de las filas (identificadores de clientes, fechas).
 - ② **Columns:** Etiquetas de las variables (Edad, Salario, Score).
 - ③ **Data:** Los valores en sí (numpy array subyacente).
- **Tipos de datos comunes en crédito:**
 - ▶ int64: IDs, número de hijos.
 - ▶ float64: Montos de crédito, tasas de interés.
 - ▶ object/category: Estado civil, tipo de vivienda.

Creación y Carga de DataFrames

Sintaxis Básica

```
import pandas as pd

# Carga desde CSV (com n en bur de cr dito)
df_credito = pd.read_csv('cartera_clientes.csv')

# Inspección inicial
print(df_credito.head())    # Primeras 5 filas
print(df_credito.info())    # Tipos de datos y nulos
```

Limpieza de Datos: El Problema de la Realidad

En el sector financiero, los datos nunca llegan perfectos.

- **Valores Nulos (NaN):**

- ▶ *Causa:* Error de sistema, campo no obligatorio, cliente no reportado.
- ▶ *Riesgo:* Sesgo en modelos de scoring.

- **Duplicados:**

- ▶ *Causa:* Re-procesamiento de archivos batch, doble entrada manual.
- ▶ *Riesgo:* Inflar la exposición al riesgo o contar doble una deuda.

Manejo de Valores Nulos

Estrategias para tratar datos faltantes en crédito:

2. Imputación

1. Eliminación

- Útil si la fila carece de ID o Target.
- `df.dropna()`

- Llenar con media/mediana (ingresos).
- Llenar con "Desconocido" (categóricas).
- `df.fillna(value)`

```
# Ejemplo: Imputar ingresos con la mediana
mediana_ingreso = df['ingreso'].median()
df['ingreso'] = df['ingreso'].fillna(mediana_ingreso)
```

Manejo de Duplicados

Identificar y eliminar registros repetidos es crucial para el reporte regulatorio.

```
# Verificar duplicados totales
duplicados = df.duplicated().sum()

# Eliminar duplicados basandonos en ID de credito
# keep='last' mantiene el registro mas reciente (
    actualizaci n)
df_clean = df.drop_duplicates(subset=['id_credito'],
                              keep='last')
```

Manejo de data heterogénea

En el análisis univariado, nos enfocamos en una sola variable. Para detectar datos ruidosos, se definen intervalos de confianza donde la mayoría de los datos "normales" deberían caer. Observaciones fuera de estos intervalos se consideran potencialmente ruidosas y podrían requerir limpieza, como eliminación, imputación o investigación adicional.

- Distribución Similar a Normal: Para distribuciones que se aproximan a una normal (gaussiana), el intervalo contiene aproximadamente el 96 % de los datos, donde μ es la media y σ es la desviación estándar.

$$[\mu - 2\sigma, \mu + 2\sigma]$$

Manejo de data heterogénea

En el manejo de datos ruidosos, este intervalo se usa para identificar outliers: por ejemplo, en un conjunto de datos de temperaturas, valores fuera de este rango podrían indicar sensores defectuosos. Una regla común es el z-score: si $|z| > 2$ (donde $z = (x - \mu)/\sigma$), el punto es sospechoso de ser ruido. Para un umbral más estricto, se usa 3σ (99.7 % de los datos), pero 2σ es útil para detección inicial en datasets con ruido moderado. En el caso general (sin asumir normalidad), el teorema de Chebyshev establece que, para cualquier $\gamma > 1$, el intervalo

$$[\mu - \gamma\sigma, \mu + \gamma\sigma]$$

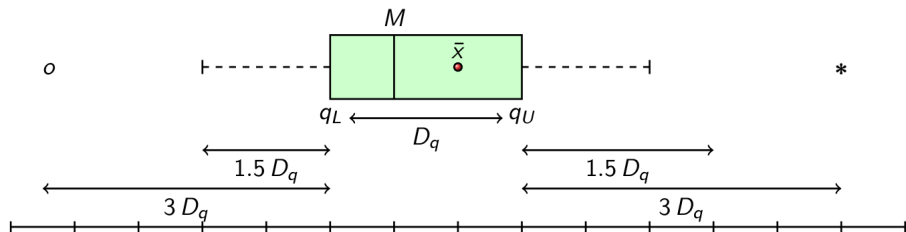
contiene al menos la proporción $1 - 1/\gamma^2$ de las observaciones.

Manejo de data heterogénea

Este teorema es conservador y aplica a cualquier distribución, lo que lo hace ideal para datos ruidosos con distribuciones desconocidas o sesgadas. Por ejemplo, si $\gamma = 2$, al menos el 75 % de los datos están dentro del intervalo (aunque en normales es 95 %). En práctica, para manejar ruido:

- Elige γ basado en la tolerancia al ruido (e.g., $\gamma = 3$ garantiza al menos 89 % dentro).
- Marca observaciones fuera como ruidosas; útil en finanzas para detectar transacciones anómalas sin asumir normalidad.
- Ventaja: Robusto a distribuciones no normales, pero menos preciso que métodos paramétricos.

Box-plot



$$D_q = q_U - q_L = q_{0.75} - q_{0.25}$$

$$\text{internal lower edge} = q_L - 1.5 D_q$$

$$\text{external lower edge} = q_L - 3 D_q$$

Definición del Caso Práctico

Objetivo: Generar un reporte de calidad de cartera.

- **Dataset:** Datos simulados de 10,000 créditos.
- **Variables:** `id_cliente`, `monto`, `dias_mora`, `score_buro`.
- **Reglas de Negocio:**
 - ▶ **Cliente Cumplido:** Días de mora < 30 .
 - ▶ **Cliente Moroso:** Días de mora ≥ 90 .
 - ▶ **En Observación:** Días de mora entre 30 y 89.

Filtrado de Clientes (Boolean Indexing)

Usamos máscaras booleanas para segmentar la cartera.

```
# Filtro de Morosos
filtro_mora = df['dias_mora'] >= 90
morosos = df[filtro_mora]

# Filtro de Cumplidos
filtro_cumplidos = df['dias_mora'] < 30
cumplidos = df[filtro_cumplidos]

# C lculo de tasa de incumplimiento
tasa_mora = len(morosos) / len(df) * 100
```


Reporte de Calidad de Datos

Antes de modelar riesgo, debemos reportar la salud de la base.

- **Integridad:** % de Nulos por columna.
- **Unicidad:** % de IDs duplicados.

```
def reporte_calidad(df):  
    nulos = df.isnull().mean() * 100  
    duplicados = df.duplicated().sum()  
    print(f"Duplicados_totales: {duplicados}")  
    print("Porcentaje_de_nulos:\n", nulos)
```

Estructura OHLCV

Formato estándar para datos de mercado (acciones, cripto, forex).

Componente	Descripción
Open	Precio de apertura del periodo.
High	Precio máximo alcanzado.
Low	Precio mínimo alcanzado.
Close	Precio de cierre (el más importante para análisis).
Volume	Cantidad de activos negociados.

Índices Temporales (DatetimeIndex)

En finanzas, el índice **debe** ser el tiempo.

- Permite *slicing* inteligente: `df['2023-01']`
- Facilita el resampling (pasar de velas de 1min a 1h).
- **Importante:** Manejar zonas horarias y días feriados (gaps).

Valores Nulos y Outliers en Finanzas

Valores Nulos

Suelen ocurrir en días feriados si forzamos un rango de fechas continuo.

- **Solución:** `ffill()` (Forward Fill). Asume que el precio se mantiene hasta nueva información.

Outliers (Valores Atípicos)

- *Flash Crash*: Caídas abruptas por error algorítmico.
- *Bad Ticks*: Errores del proveedor de datos (precio 0 o infinito).
- Se detectan con Desviación Estándar Móvil.

El método .shift()

Desplaza los datos un número n de periodos. Esencial para calcular retornos.

$$Retorno_t = \frac{Precio_t}{Precio_{t-1}} - 1$$

```
# Desplaza el precio 1 d a hacia adelante
df['precio_ayer'] = df['Close'].shift(1)

# Diferencia absoluta
df['delta'] = df['Close'] - df['Close'].shift(1)
```

El método `.pct_change()`

Calcula el cambio porcentual inmediato.

- Automatiza la fórmula: $(P_t - P_{t-1})/P_{t-1}$

```
# Retorno diario
df['retorno_diario'] = df['Close'].pct_change()

# Retorno acumulado (interés compuesto)
df['retorno_acumulado'] = (1 + df['retorno_diario']).cumprod()
```

El método `.rolling()`

Ventanas móviles para análisis técnico (Medias Móviles, Volatilidad).

- **SMA (Simple Moving Average):** Suaviza la serie.
- **Volatilidad:** Desviación estándar móvil.

```
# Media móvil de 20 días (SMA 20)
df['SMA_20'] = df['Close'].rolling(window=20).mean()

# Volatilidad de 20 días
df['Volatilidad'] = df['Close'].rolling(window=20).std()
```

Objetivo del Laboratorio

Pregunta de Negocio: ¿Qué activo rindió más en el último año: Apple, Bitcoin o un ETF del S&P 500?

El Problema de la Escala:

- Bitcoin $\approx$ \$40,000
- Apple $\approx$ \$180
- **Solución:** Normalización Base 100.

$$PrecioNorm_t = \frac{Precio_t}{Precio_0} \times 100$$

Paso 1: Generación de Datos

En un entorno real usaríamos yfinance. Aquí simulamos series para el ejercicio.

```
# Crear DatetimeIndex
fechas = pd.date_range(start='2023-01-01', periods
                        =252, freq='B')

# Simular precios
# (Código detallado en el notebook de práctica)
precios_aapl = ...
precios_btc = ...
```

Paso 2: Normalización

Aplicamos la normalización para que todas las series empiecen en el mismo punto (100).

```
# Seleccionar solo columnas de cierre
df_cierre = df_mercado[['AAPL', 'BTC', 'SPY']]

# Normalizar: (Precio / Primer Precio) * 100
df_norm = (df_cierre / df_cierre.iloc[0]) * 100

# Verificación
print(df_norm.head(1))

# Todas deben valer 100.0
```

Paso 3: Visualización

Generar un gráfico comparativo.

```
import matplotlib.pyplot as plt

df_norm.plot(figsize=(10, 6))
plt.title('Rendimiento Comparativo (Base 100)')
plt.ylabel('Retorno Normalizado')
plt.axhline(100, color='k', linestyle='--', alpha=0.5)
plt.show()
```